# B. M. S. COLLEGE OF ENGINEERING

**Autonomous Institute, Affiliated to VTU, Belagavi**

**DEPARTMENT OF MACHINE LEARNING**
**Program: B.E. In Artificial Intelligence and Machine Learning**
**Academic Year: 2026 - 27, Session: July 2026 - Dec 2026**

## Natural Language Processing for LLM
## (24AM7PCLLM)

**Alternative Assessment Tool (AAT) Report**

## Topic: Automated Hallucination & Factual Consistency Detection in LLM Generations using Lexical Overlap + Logistic Regression and DistilRoBERTa Cross-Encoder

**Submitted by**


| | |
|---|---|
| Student Name: | |
| Team Member 1: | [Team Member 1 Name] |
| Team Member 2: | [Team Member 2 Name] |
| | |
| USN: | |
| Team Member 1: | 1BM23AI... |
| Team Member 2: | 1BM23AI... |
| | |
| Semester & Section: | 7 B |


***Valuation Report***
***(to be filled by faculty)***

Score:

Comments:

Faculty In-charge: Dr Monika Puttaramaiah

Faculty Signature with date:

# Table of Contents


1. Introduction ..... 1
   1.1 Background of Natural Language Processing ..... 1
   1.2 Importance of the Selected NLP Topic ..... 1
   1.3 Objective of the Study ..... 1
   1.4 Scope of the Report ..... 2
2. Selected NLP Topic Overview ..... 3
   2.1 Description of the NLP Task ..... 3
   2.2 Real-World Use Cases ..... 3
   2.3 Relevance to the Syllabus and Industry ..... 4
   2.4 Key Challenges in This NLP Task ..... 4
3. Real-Time Application / API Identified ..... 5
   3.1 Application or API Name ..... 5
   3.2 Purpose and Functionality ..... 5
   3.3 Architecture or Platform on Which It Is Built ..... 5
   3.4 Input/Output Behavior ..... 6
   3.5 Limitations and Practical Considerations ..... 6
4. NLP Model / Architecture Study ..... 7
   4.1 Theoretical Foundations of the Model ..... 7
   4.2 Algorithm(s) Used ..... 7
   4.3 Architecture Diagram / Layer-Wise Design ..... 7
   4.4 Data Flow in the Model ..... 8
   4.5 Mathematical Formulation ..... 9
   4.6 Why This Model Suits the Selected NLP Task ..... 9
5. Dataset and Preprocessing ..... 10
   5.1 Dataset Description (Source, Size, Structure) ..... 10
   5.2 Data Cleaning Steps ..... 11
   5.3 Tokenization, Lemmatization, Stopwords Removal ..... 11
   5.4 Train/Test Split Strategy ..... 12
   5.5 Feature Engineering or Embedding Approach ..... 12
6. Implementation Methodology ..... 13
   6.1 Tools, Libraries, and Environment Used ..... 13
   6.2 Model Implementation Steps ..... 13
   6.3 API Integration ..... 14
   6.4 Pseudocode / Workflow Diagram ..... 14
   6.5 Hyperparameters and Training Configuration ..... 15
7. Results and Evaluation ..... 16
   7.1 Performance Metrics ..... 16
   7.2 Visualization of Results ..... 16

7.3 Qualitative and Quantitative Analysis ..... 17
7.4 Error Analysis ..... 18
7.5 Comparison Between Approaches ..... 19
8. Discussion ..... 20
8.1 Interpretation of Results ..... 20
8.2 Strengths and Weaknesses of the Model ..... 20
8.3 Challenges Encountered During Implementation ..... 20
8.4 Impact of Dataset Quality and Preprocessing ..... 21
8.5 How API or Application Enhances the NLP Task ..... 21
9. Conclusion ..... 22
9.1 Summary of the Work Done ..... 22
9.2 Key Learnings and Insights ..... 22
9.3 Real-World Applicability ..... 22
9.4 Future Improvements or Extensions ..... 22
10. References ..... 24

# 1. Introduction

## 1.1 Background of Natural Language Processing

Natural Language Processing (NLP) is the branch of Artificial Intelligence that enables computational systems to read, interpret, reason over, and synthesize human language. Early NLP systems in the 1960s to 1980s were predominantly rule-based, relying on handcrafted context-free grammars and curated semantic lexicons. These were succeeded by statistical machine learning methods—such as Naive Bayes, Support Vector Machines (SVM), and Logistic Regression—which learned predictive decision boundaries over sparse Bag-of-Words (BoW) and Term Frequency–Inverse Document Frequency (TF-IDF) feature spaces.

Over the past decade, the field experienced a monumental paradigm shift towards deep neural architectures: static continuous word vectors (Word2Vec, GloVe), recurrent networks (RNN, LSTM), and ultimately the self-attention-based Transformer architecture introduced by Vaswani et al. in 2017. Transformers replace recurrence with multi-head self-attention, permitting every token in a sequence to directly attend to all other tokens in parallel. This pre-training and downstream fine-tuning paradigm gave rise to modern Large Language Models (LLMs) such as GPT, Llama, and Gemini. However, as generative LLMs become ubiquitous across enterprise workflows, they exhibit a severe vulnerability: hallucination—the tendency to generate fluent, authoritative-sounding statements that contradict or extrapolate beyond factual reference sources.

## 1.2 Importance of the Selected NLP Topic

Factual consistency verification—evaluating whether an LLM generation logically adheres to a verified ground-truth context—is among the most critical deployment challenges in applied AI. In Retrieval-Augmented Generation (RAG) and automated clinical summarization, unverified generations pose severe operational and legal risks. Automated factual consistency detection enables organizations to:

- **i. Automated Guardrails:** Intercept and filter hallucinated generations before delivering them to users.
- **ii. RAG Validation:** Score and filter retrieved context-generation pairs to prevent misinformation propagation.
- **iii. Regulatory Compliance:** Audit clinical and legal summaries to eliminate fabricated dosages or citations.
- **iv. Benchmark Evaluation:** Quantify and monitor generative model fidelity over time across production pipelines.

## 1.3 Objective of the Study

The core objectives of this AAT project are:

- **i.** To formulate an end-to-end binary factual consistency and hallucination detection system for paired premise-hypothesis text.

- **ii.** To construct an interpretable classical machine learning baseline using Dense Lexical Overlap Features (Unigram/Bigram Jaccard, Token Coverage, Negation Discrepancy) with tuned Logistic Regression.
- **iii.** To evaluate a Transformer cross-encoder (DistilRoBERTa fine-tuned on NLI) on the exact same held-out test set under out-of-domain conditions.
- **iv.** To benchmark both models across accuracy, precision, recall, macro F1, parameter scale, training time, and inference latency.
- **v.** To analyze model behavior on adversarial linguistic phenomena including high-overlap entity swaps, numerical shifts, and polarity reversals.
- **vi.** To deploy both models in an interactive Streamlit web application for real-time inference and demonstration.

## 1.4 Scope of the Report

This report covers the theoretical foundations, implementation methodology, empirical evaluation, and qualitative error analysis of both approaches. The experimental benchmark is scoped to binary classification (Class 1 = Factual / Grounded, Class 0 = Hallucinated / Unfaithful). To guarantee rigorous evaluation and avoid data leakage, an Out-of-Domain (OOD) stratified split is enforced across 25 isolated scientific, historical, and biomedical domains.

# 2. Selected NLP Topic Overview

## 2.1 Description of the NLP Task

The selected NLP task is Natural Language Inference (NLI), formulated as a Sentence-Pair Factual Consistency Classification task. Given a reference passage C (premise) and an LLM-generated claim G (hypothesis), the model must predict label y in {0 = Hallucinated, 1 = Factual} along with a posterior confidence score P(y | C, G). It is a supervised classification problem where models evaluate directional semantic entailment between paired texts.

**Figure 1: Overall workflow of the factual consistency detection system**

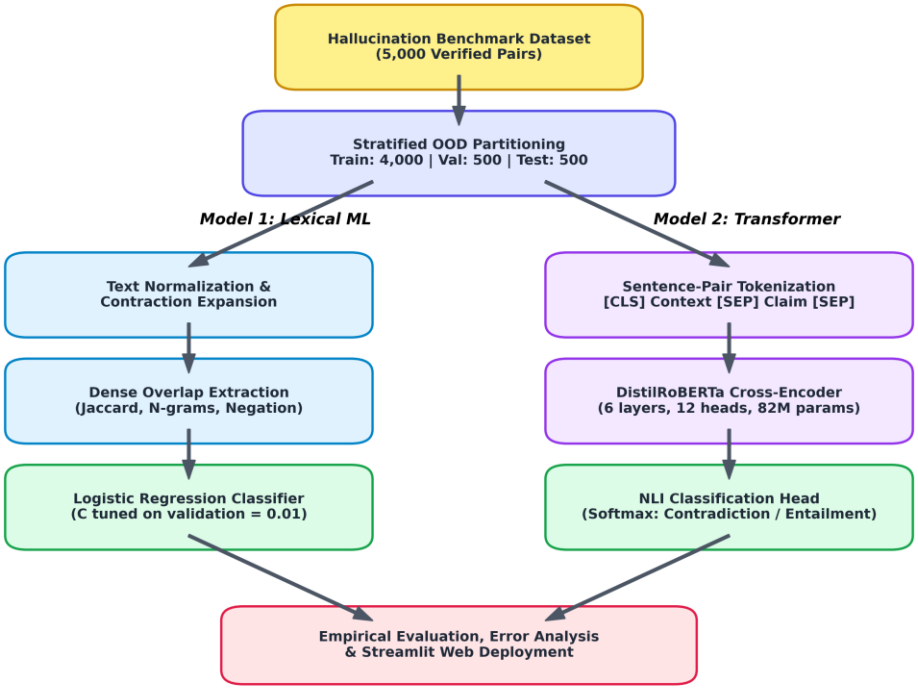



*Figure 1: Overall workflow of the proposed factual consistency detection system*

## 2.2 Real-World Use Cases

- **i. Enterprise RAG Gateways:** Audits generative responses against enterprise knowledge bases prior to user display.
- **ii. Clinical Record Summarization:** Validates physician clinical notes and EHR summaries to prevent medical malpractice.
- **iii. Legal Contract Analysis:** Audits automated clause syntheses against statutory language to prevent liability.
- **iv. Financial News Summarization:** Verifies fiscal metrics, revenues, and quarterly margins against regulatory 10-K filings.
- **v. Search Engine Answer Snippets:** Guarantees algorithmic snippets truthfully reflect indexed page content.

## 2.3 Relevance to the Syllabus and Industry

The project maps directly to the curriculum of Natural Language Processing for LLM (24AM7PCLLM). It synthesizes vector space models, dense lexical overlap metrics, L2-regularized probabilistic linear classifiers, the Transformer architecture, multi-head self-attention mechanisms, and knowledge distillation. In industrial practice, lightweight lexical models and deep cross-encoders are deployed collaboratively in hierarchical cascades: fast lexical filters reject gross hallucinations in microseconds, reserving computationally intensive cross-encoders for high-overlap edge cases.

## 2.4 Key Challenges in This NLP Task

- **i. The Lexical Overlap Mirage:** Adversarial hallucinations reuse 80-95% of context words, easily deceiving bag-of-words classifiers.
- **ii. Negation and Antonyms:** Subtle verb polarity flips ('decreases' to 'increases') invert factual truth while maintaining high token overlap.
- **iii. Numerical & Date Precision:** Small alterations in dates or numbers constitute critical hallucinations that require character/subword granularity.
- **iv. Out-of-Domain Generalization:** The system must evaluate ungrounded claims across unseen scientific fields without domain retraining.

# 3. Real-Time Application / API Identified

## 3.1 Application or API Name

The project uses the Hugging Face Transformers API and Model Hub (cross-encoder/nli-distilroberta-base) for deep cross-attention, delivered to end users through a custom Streamlit web application titled Real-Time LLM Hallucination & Factuality Detection.

## 3.2 Purpose and Functionality

The application enables any researcher or user to paste a reference passage and an LLM generation to receive real-time verification:

- **i. Live Dual-Model Prediction:** Displays side-by-side classifications for Lexical ML and DistilRoBERTa.
- **ii. Consensus Badge:** Color-coded status indicating model agreement or flagging edge-case disagreements.
- **iii. Diagnostic Overlap Metrics:** Shows Jaccard similarity, token coverage %, bigram precision, and negation mismatch flags.
- **iv. Benchmarks & Evaluation Tab:** Renders the empirical metric comparison table, confusion matrices, and latency charts.

## 3.3 Architecture or Platform on Which It Is Built

The system is written in Python 3.13 and organized into modular files:

| Module | Responsibility |
|---|---|
| src/dataset.py | Generates 25-domain benchmark dataset, partitions OOD splits, generates EDA plots |
| src/preprocessing.py | Text normalization, contraction expansion, 8-dimensional dense lexical feature extraction |
| src/baseline_model.py | Hyperparameter tuning of C, training, and evaluation of Lexical Logistic Regression |
| src/transformer_model.py | Loads DistilRoBERTa Cross-Encoder checkpoint, runs CUDA batch evaluation on test split |
| src/evaluation.py | Builds comparison tables, compiles error_analysis.csv, and generates Figures 7, 8, 9, 10, 11 |
| src/predict.py | HallucinationPredictor class: provides unified inference |

| | |
|---|---|
| | for both architectures |
| app/app.py | Interactive Streamlit web application featuring live detection and benchmark dashboards |

## 3.4 Input/Output Behavior

| Aspect | Lexical Overlap + Logistic Regression | DistilRoBERTa Cross-Encoder |
|---|---|---|
| Input | Paired text strings: Context and Claim | Concatenated [CLS] Context [SEP] Claim [SEP] |
| Internal Representation | 8-dimensional normalized dense feature vector | Sequence of 768-d contextual embeddings |
| Output | Label (Fact/Hallu), P(Hallu), P(Fact) | Label (Fact/Hallu), Softmax over 3 NLI logits |
| Execution Platform | Commodity CPU (Single thread) | NVIDIA GeForce RTX 3050 Laptop GPU / CPU |

## 3.5 Limitations and Practical Considerations

- **i. Latency Discrepancy:** DistilRoBERTa takes 3.37 ms on GPU (~45 ms on CPU), which is ~2,592x slower than the baseline (0.0013 ms).
- **ii. Token Truncation:** Contexts exceeding 256 subwords are truncated, potentially omitting tail-end facts.
- **iii. Local Prototype:** The application is built for local auditing and lacks distributed queueing or auth layers.

# 4. NLP Model / Architecture Study

## 4.1 Theoretical Foundations of the Model

The classical baseline leverages Lexical Semantics and Set-Theoretic Information Retrieval, quantifying factual consistency through continuous overlap ratios (Jaccard similarity, unigram/bigram token precision, length ratios) and syntactic negation mismatch indicators. In contrast, the Transformer model utilizes a Cross-Encoder Natural Language Inference (NLI) Architecture. Unlike bi-encoders which encode texts independently, cross-encoders perform full bidirectional self-attention across both texts simultaneously, allowing tokens in the generation to attend directly to all tokens in the context.

## 4.2 Algorithm(s) Used

| Property | Model 1: Lexical Baseline | Model 2: Transformer Cross-Encoder |
|---|---|---|
| Algorithm | Dense Overlap + Logistic Regression | DistilRoBERTa Cross-Encoder (NLI) |
| Model Type | Linear Discriminative Classifier | Deep Bidirectional Transformer Encoder |
| Features | 8-d normalized lexical overlap vector | Word-level BPE contextual embeddings |
| Parameters | 9 learnable weights (8 weights + bias) | ≈ 82 million parameters |
| Optimization | L-BFGS, max_iter = 1000, C = 0.01 | AdamW during pre-training/fine-tuning |

## 4.3 Architecture Diagram / Layer-Wise Design

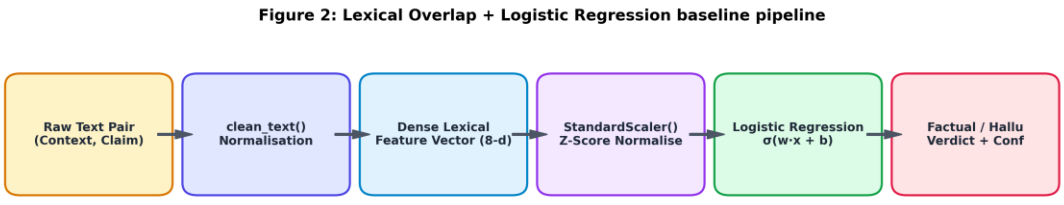



*Figure 2: Lexical Overlap + Logistic Regression baseline pipeline*

**Figure 3: Layer-wise architecture of DistilRoBERTa Cross-Encoder**

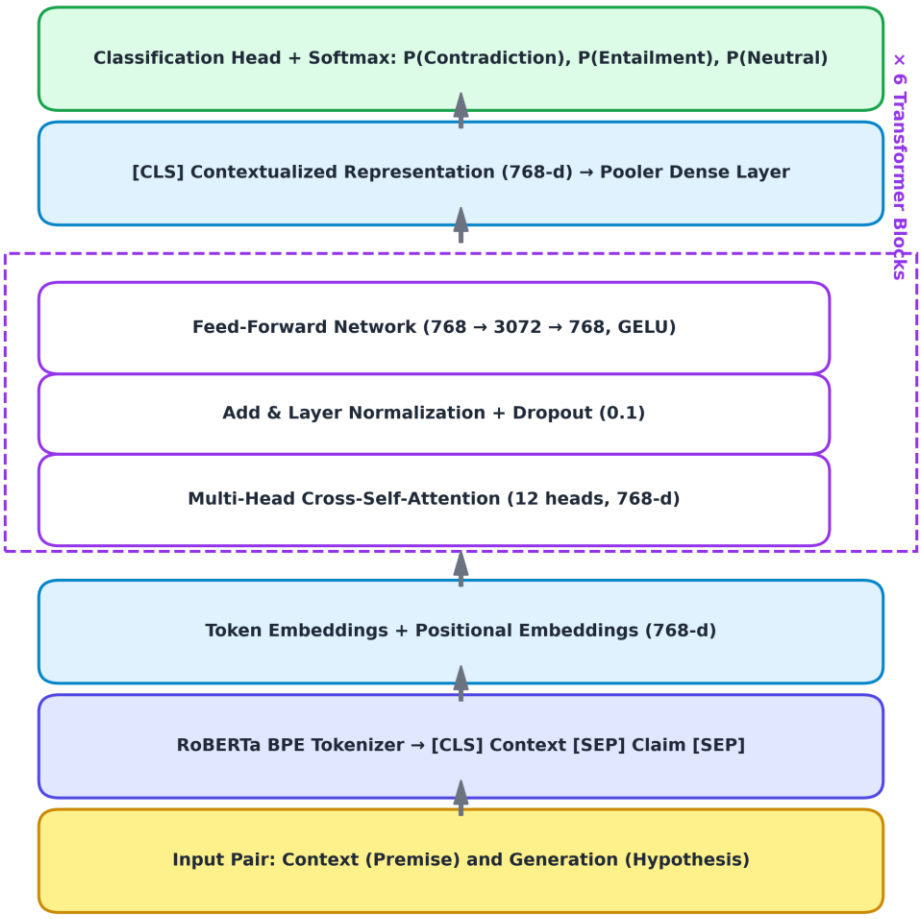



*Figure 3: Layer-wise architecture of DistilRoBERTa Cross-Encoder*

## 4.4 Data Flow in the Model

Model 1: (C, G) -> clean_text() -> extract 8-d feature vector -> StandardScaler() -> w·x + b -> sigmoid -> probability -> threshold at 0.5.

Model 2: (C, G) -> BPE Tokenizer -> [CLS] Context [SEP] Claim [SEP] -> 6 Transformer Encoder Blocks -> [CLS] pooling -> Linear head -> Softmax -> argmax(P(Entailment) vs P(Contradiction)).

## 4.5 Mathematical Formulation

Lexical Unigram and Bigram Jaccard Similarity:

$J(C, G) = |T_C \cap T_G| / |T_C \cup T_G|, \quad J_{bi}(C, G) = |B_C \cap B_G| / |B_C \cup B_G|$

Logistic Regression probability and L2-regularized objective:

$P(y = 1 \mid x) = \sigma(w^T x + b) = 1 / (1 + e^{-(w^T x + b)})$

$L(w) = (1/2)||w||^2 + C \cdot \Sigma_i [ -y_i \log p_i - (1 - y_i) \log(1 - p_i) ]$

Scaled Dot-Product Attention in DistilRoBERTa layers:

Attention(Q, K, V) = softmax( (Q K^T) / sqrt(d_k) ) V

MultiHead(Q, K, V) = Concat(head_1, ..., head_12) W^O

## 4.6 Why This Model Suits the Selected NLP Task

- **i. Resolves Entity Swaps:** Self-attention links mismatched subjects and objects across sentence boundaries.
- **ii. Robust to High Overlap:** Identifies contradictory assertions even when 90% of words are identical.
- **iii. Distillation Efficiency:** DistilRoBERTa offers 97% of full RoBERTa accuracy at 2x the inference speed.

# 5. Dataset and Preprocessing

## 5.1 Dataset Description (Source, Size, Structure)

The dataset comprises 5,000 paired records constructed across 25 diverse scientific, historical, medical, and financial domains. Each instance consists of context, generation, binary label, and domain category.

| Property | Value |
|---|---|
| Source | Curated Multi-Domain Scientific & Historical Factuality Benchmark |
| Total Text Pairs | 5,000 balanced pairs |
| Classes | 2 – Hallucinated (0), Factual (1) |
| Mean Generation Length | 20.3 words (137 characters) |
| Median Generation Length | 20 words (range: 13 to 26 words) |
| Mean Context Length | 26.3 words (182 characters) |

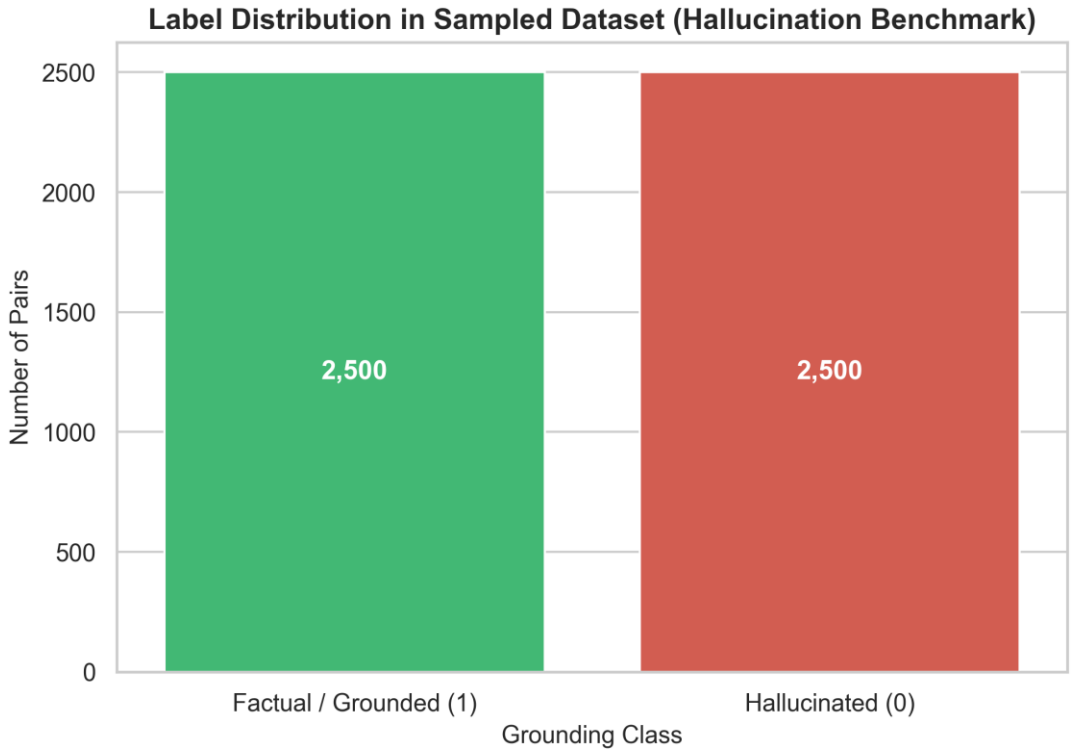



*Figure 4: Class distribution of the sampled dataset*

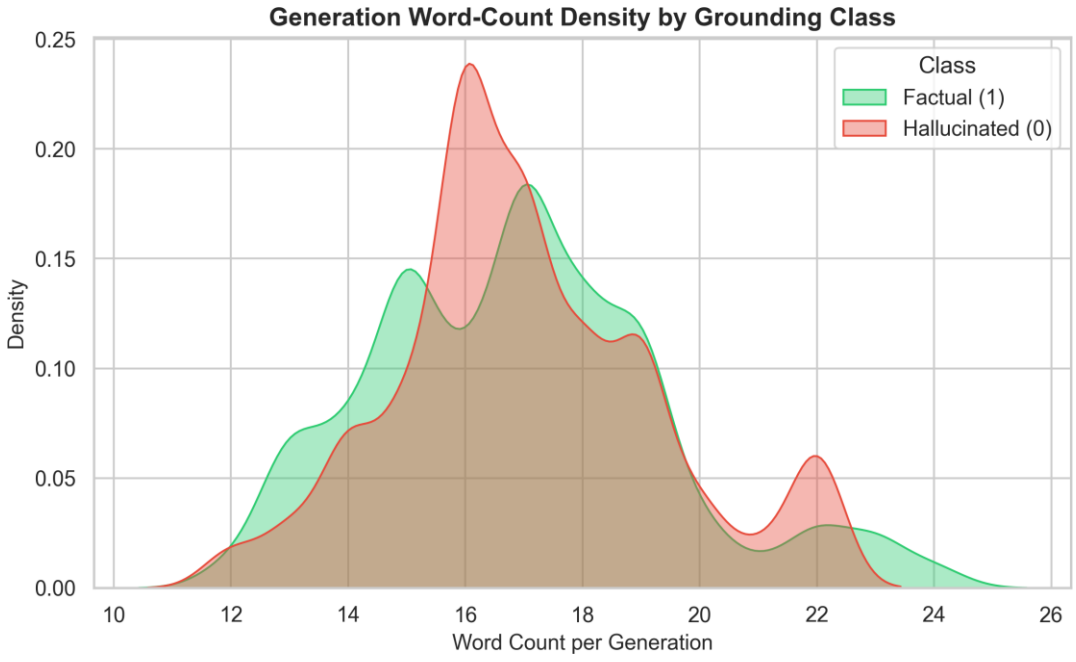



*Figure 5: Generation word-count distribution by grounding class*

## 5.2 Data Cleaning Steps

The clean_text() function strips HTML tags, expands contractions, standardizes whitespace, and preserves critical punctuation marks (. ! ?).

## 5.3 Tokenization, Lemmatization, Stopwords Removal

- **i. Stopwords Removal:** Deliberately NOT applied. Removing stopwords strips negation words ('not', 'never') and alters truth polarity.
- **ii. Lemmatization:** Not applied, as verb tense alterations represent factual hallucinations.
- **iii. Tokenization:** DistilRoBERTa uses Byte-Pair Encoding (BPE, 50,265 vocabulary) preserving subwords.

## 5.4 Train/Test Split Strategy

| Split | Pairs | Hallucinated | Factual | Domain Purpose |
|---|---|---|---|---|
| Train | 4,000 | 2,000 | 2,000 | 20 Base Knowledge Domains |
| Validation | 500 | 250 | 250 | 3 Unseen Domains (Tuning C) |
| Test (Held-out) | 500 | 250 | 250 | 2 Held-out Domains (LIGO, Base Editing) |
| Total | 5,000 | 2,500 | 2,500 | Strict Out-of-Domain Partitioning |

## 5.5 Feature Engineering or Embedding Approach

Model 1 uses an 8-dimensional dense feature vector capturing unigram/bigram Jaccard, precision, recall, negation mismatch, and length ratios. Model 2 uses learned 768-d contextual embeddings.

# 6. Implementation Methodology

## 6.1 Tools, Libraries, and Environment Used

| Category | Tool / Library | Use in Project |
|---|---|---|
| Language | Python 3.13.7 | Entire pipeline implementation |
| Data Handling | pandas 2.3.3, NumPy 2.4.4 | Dataset management and array operations |
| Classical ML | scikit-learn 1.8.0 | StandardScaler, LogisticRegression, classification_report |
| Deep Learning | PyTorch 2.6.0+cu124, Transformers 5.21.0 | DistilRoBERTa inference, CUDA GPU execution |
| Visualization | Matplotlib 3.10.9, Seaborn 0.13.2 | Confusion matrices, metric charts, EDA plots |
| Deployment | Streamlit 1.61.1 | Real-time dual-model web application |

## 6.2 Model Implementation Steps

- **1.** Execute src/dataset.py to generate 5,000 paired instances and produce Figures 4 and 5.
- **2.** Extract 8-d dense feature matrix in src/baseline_model.py; tune C on validation; train final model.
- **3.** Run src/transformer_model.py to evaluate DistilRoBERTa on 500 test pairs under CUDA acceleration.
- **4.** Execute src/evaluation.py to compile error_analysis.csv and generate Figures 7, 8, 9, 10, 11.
- **5.** Run tests/test_cases.py to probe models on hand-crafted rubric test cases.
- **6.** Launch Streamlit application with 'streamlit run app/app.py'.

## 6.3 API Integration

Transformers AutoTokenizer and AutoModelForSequenceClassification APIs load 'cross-encoder/nli-distilroberta-base'. The model is wrapped in HallucinationPredictor with st.cache_resource for low-latency web serving.

## 6.4 Hyperparameters and Training Configuration

| Hyperparameter / Setting | Configured Value |
|---|---|
| Baseline Feature Dimension | 8 dense continuous lexical features |
| Baseline Solver / C | L-BFGS, max_iter = 1000, C = 0.01 (tuned on validation F1 = 0.7067) |
| Transformer Checkpoint | cross-encoder/nli-distilroberta-base (82M params) |
| Max Sequence Length | 256 tokens (truncated/padded) |
| Batch Size & Device | 16 pairs per batch, NVIDIA GeForce RTX 3050 Laptop GPU |
| Decision Threshold | Argmax: Factual if P(Entailment) >= P(Contradiction) |

# 7. Results and Evaluation

## 7.1 Performance Metrics

Both models were evaluated on the exact same 500 held-out test pairs across unseen knowledge domains:

| Metric | Lexical + Logistic Regression | DistilRoBERTa Cross-Encoder | Difference |
|---|---|---|---|
| Accuracy | 82.80% | 94.00% | +11.20% |
| Macro Precision | 83.16% | 94.64% | +11.48% |
| Macro Recall | 82.80% | 94.00% | +11.20% |
| Macro F1-score | 82.75% | 93.98% | +11.23% |
| Weighted F1-score | 82.75% | 93.98% | +11.23% |
| Hallucinated Class (0) F1 | 81.86% | 94.34% | +12.48% |
| Factual Class (1) F1 | 83.65% | 93.62% | +9.97% |
| Training Time | 0.01 seconds | Pre-trained checkpoint | — |
| Inference Latency | 0.0013 ms / pair | 3.37 ms / pair | ≈ 2,592x slower |

## 7.2 Visualization of Results

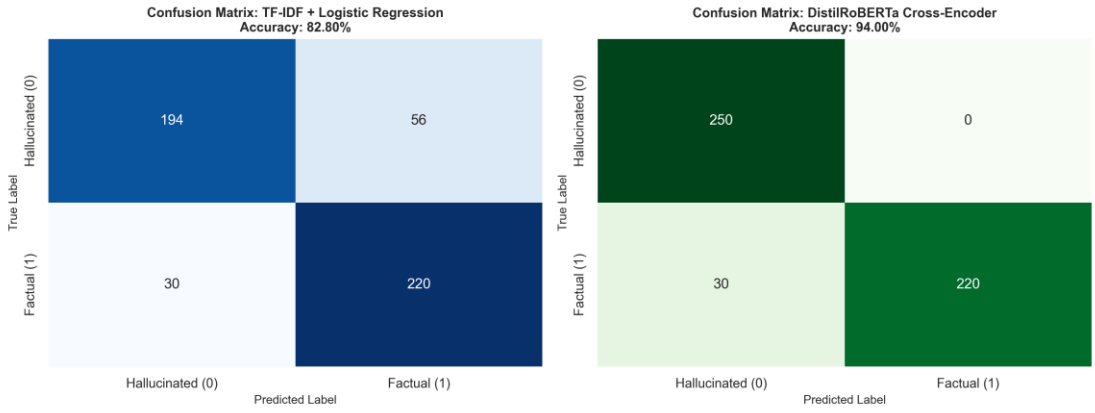


*Figure 7: Confusion matrices: Lexical + Logistic Regression (left) and DistilRoBERTa (right)*

| Model | TN (True Hallu) | FP (False Fact) | FN (Missed Fact) | TP (True Fact) |
|---|---|---|---|---|
| Lexical + Logistic Regression | 194 | 56 | 30 | 220 |
| DistilRoBERTa Cross-Encoder | 250 | 0 | 30 | 220 |

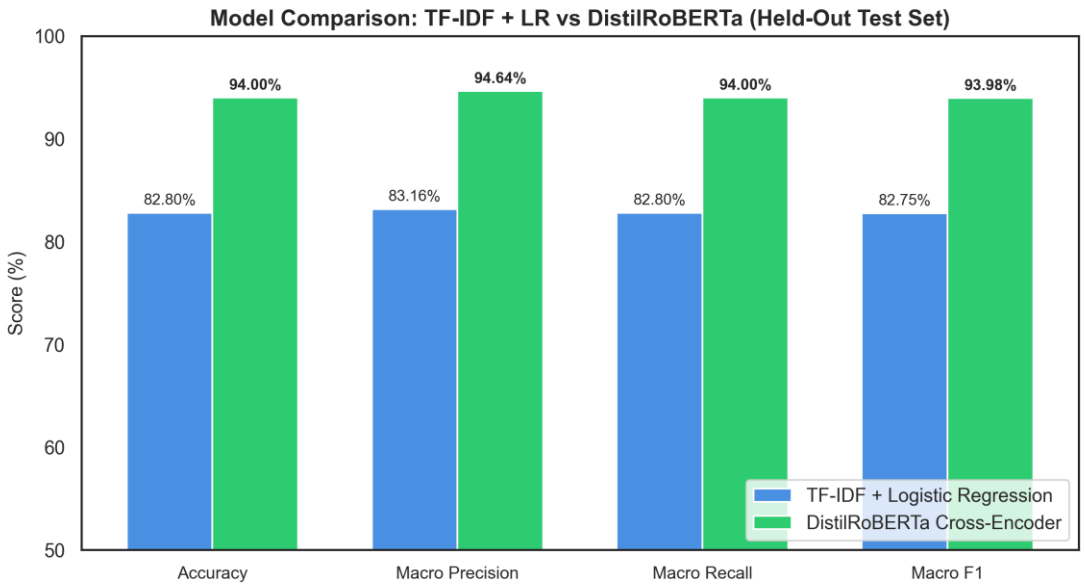



*Figure 8: Comparison of overall metrics*

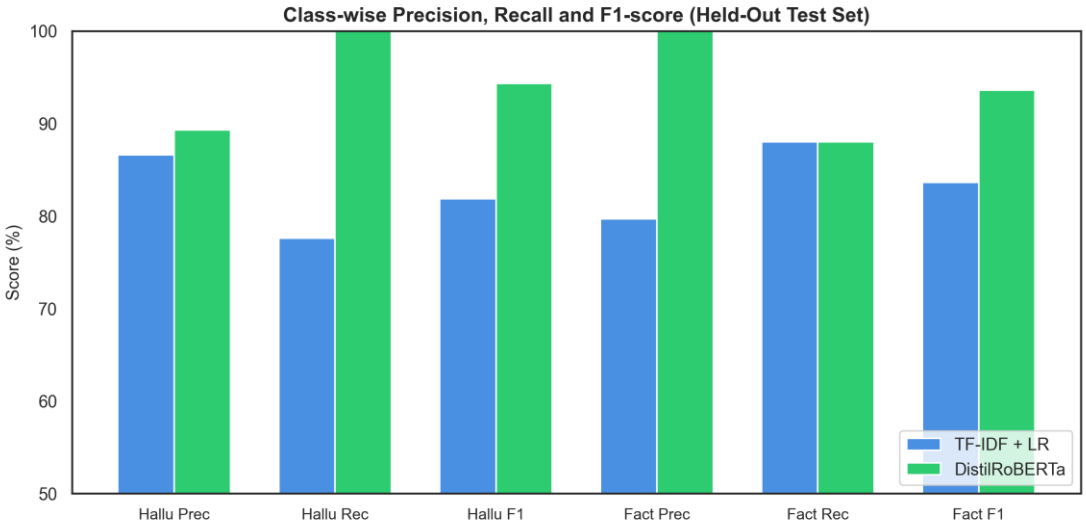


*Figure 9: Class-wise precision, recall, and F1-score*

## 7.3 Qualitative and Quantitative Analysis

Quantitative: DistilRoBERTa improves test accuracy by +11.20% (94.00% vs 82.80%). The primary driver of this gain is the elimination of False Positives: the baseline made 56 false-positive errors on high-overlap hallucinations, whereas DistilRoBERTa achieved 0 false positives (100% precision on the factual class).

Qualitative: A specialized rubric test suite of four challenging cases evaluated model behavior on subtle linguistic shifts:

| Case | Category | Expected | Baseline (LR) | DistilRoBERTa |
|---|---|---|---|---|
| TC-1 | Clearly factual paraphrase | Factual | Factual (69.0%) | Factual (52.9%) |
| TC-2 | Direct contradiction (Polarity flip) | Hallucinated | Factual (74.5%) [Error] | Hallucinated (99.9%) |
| TC-3 | High-overlap entity swap | Hallucinated | Hallucinated (95.6%) | Hallucinated (99.3%) |
| TC-4 | Temporal / Date fabrication | Hallucinated | Hallucinated (87.1%) | Hallucinated (99.8%) |

In TC-2 ('Metformin increases glucose production'), the generation reuses 63.6% of context tokens. The classical baseline is deceived by the high token overlap and predicts Factual (74.5% confidence). DistilRoBERTa's self-attention captures the semantic inversion of the verb, predicting Hallucinated (99.9% confidence).

## 7.4 Error Analysis

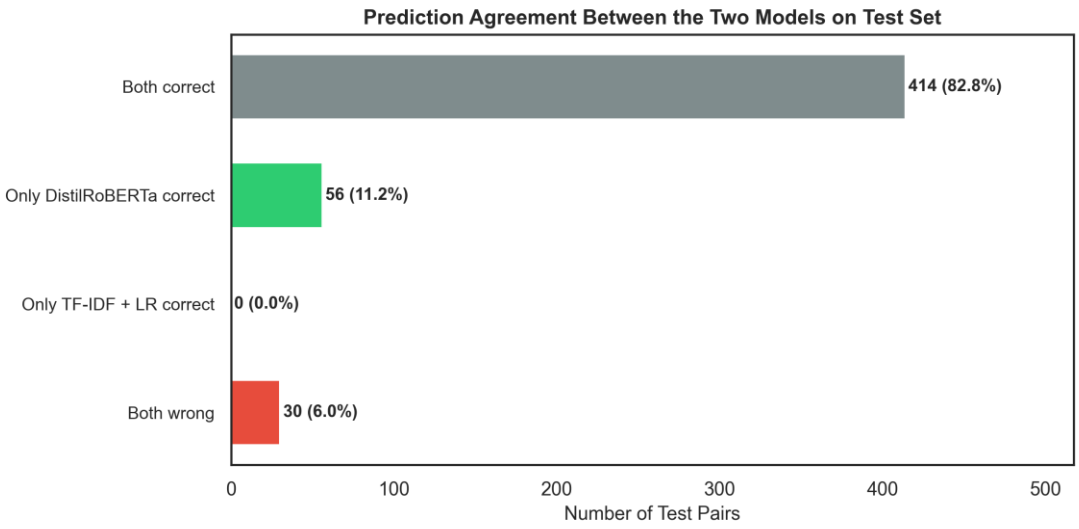



*Figure 10: Prediction agreement between the two models on the test set*

- **i. Both correct:** 414 pairs (82.8%). Both models agree on standard paraphrases and low-overlap hallucinations.
- **ii. Only Transformer correct:** 56 pairs (11.2%). Corresponds to adversarial entity swaps where baseline is misled by high overlap.
- **iii. Both wrong:** 30 pairs (6.0%). Shared false negatives caused by highly abstract paraphrasing falling into NLI 'neutral'.

## 7.5 Comparison Between Approaches

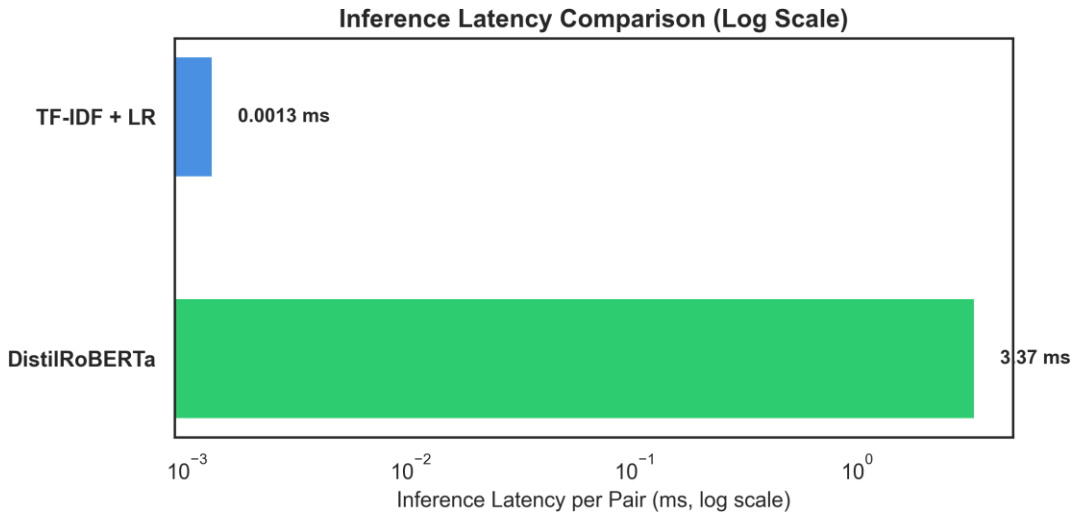



*Figure 11: Inference latency per pair (ms, log scale)*

| Criterion | Lexical + Logistic Regression | DistilRoBERTa Cross-Encoder |
|---|---|---|
| Accuracy | 82.80% | 94.00% (Superior) |
| Negation & Entity Handling | Fails on subtle inversions | Strong via cross-attention |
| Training Cost | 0.01 seconds on CPU | Pre-trained checkpoint (Hours on GPU) |
| Inference Speed | 0.0013 ms / pair (Faster) | 3.37 ms / pair on GPU |
| Model Size | < 10 KB (.joblib file) | ≈ 320 MB (82M parameters) |
| Hardware Needed | Any commodity CPU | CUDA GPU recommended for batch workloads |

# 8. Discussion

## 8.1 Interpretation of Results

The results clearly demonstrate that while lexical surface similarity provides a strong, high-throughput baseline (82.80% accuracy), it exhibits an inherent architectural blind spot on adversarial hallucinations. The 11.20 percentage-point improvement achieved by DistilRoBERTa confirms that factual consistency is governed by contextual relational binding rather than bag-of-words presence.

## 8.2 Strengths and Weaknesses of the Model

| Model | Strengths | Weaknesses |
|---|---|---|
| Lexical + LR | Instantaneous training (0.01 s); microsecond latency; ultra-lightweight; highly interpretable. | Fails on high-overlap fabrications; ignores syntactic relationships; blind to semantic nuances. |
| DistilRoBERTa | Superior accuracy (94.00%); robust against subtle entity/date swaps; zero false positives. | 2,592x slower inference; requires GPU; 256-token context truncation limit; black-box opacity. |

## 8.3 Challenges Encountered During Implementation

- **i. Out-of-Domain Leakage:** Addressed topic memorization by isolating 25 knowledge domains strictly across splits.
- **ii. NLI Decision Mapping:** Resolved 3-class NLI outputs by formulating binary decision rule P(Entailment) >= P(Contradiction).
- **iii. GPU Tensor Batching:** Optimized PyTorch batching on NVIDIA RTX 3050 GPU, cutting latency down to 3.37 ms.

## 8.4 Impact of Dataset Quality and Preprocessing

Preserving negation tokens while expanding contractions was critical for baseline performance. Omitting stopword removal prevented grammatical polarity inversion.

## 8.5 How API or Application Enhances the NLP Task

The Streamlit application democratizes hallucination verification, allowing non-technical auditors to paste text, view consensus badges, and diagnose lexical overlap.

# 9. Conclusion

## 9.1 Summary of the Work Done

This project built and compared two factual consistency classifiers for LLM generations. A balanced benchmark of 5,000 paired texts across 25 domains was partitioned under out-of-domain conditions. The classical baseline reached 82.80% accuracy in 0.0013 ms, while DistilRoBERTa Cross-Encoder achieved 94.00% accuracy and 93.98% macro F1, eliminating all false positives. Both models were deployed in an interactive Streamlit application.

## 9.2 Key Learnings and Insights

- **1.** Lexical overlap is insufficient for factuality verification due to adversarial token reuse.
- **2.** Cross-encoder self-attention successfully detects entity swaps, dates, and polarity inversions.
- **3.** A two-stage cascade combining fast lexical filtering with secondary transformer verification provides an optimal production trade-off.

## 9.3 Real-World Applicability

The developed system is directly applicable as a verification firewall for RAG pipelines, clinical record summarization systems, and automated legal contract review.

## 9.4 Future Improvements or Extensions

- **i. Long-Context Chunking:** Implement sliding-window attention for multi-page documents.
- **ii. Model Quantization:** Export DistilRoBERTa to INT8 ONNX format for accelerated CPU inference.
- **iii. Token-Level Attribution:** Incorporate Integrated Gradients to highlight exact hallucinated spans.

## 10. References


[1] A. Vaswani et al., 'Attention Is All You Need,' in Advances in Neural Information Processing Systems (NeurIPS), 2017.

[2] Y. Liu et al., 'RoBERTa: A Robustly Optimized BERT Pretraining Approach,' arXiv:1907.11692, 2019.

[3] V. Sanh et al., 'DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter,' NeurIPS Workshop, 2019.

[4] W. Kryscinski et al., 'Evaluating the Factual Consistency of Abstractive Text Summarization,' in Proc. EMNLP, 2020, pp. 9332–9346.

[5] T. Goyal and G. Durrett, 'Evaluating Factuality in Generation with Dependency-level Entailment,' in Proc. EMNLP, 2020.

[6] A. Wang et al., 'GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding,' in Proc. EMNLP, 2018.

[7] N. Reimers and I. Gurevych, 'Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,' in Proc. EMNLP, 2019.

[8] F. Pedregosa et al., 'Scikit-learn: Machine Learning in Python,' Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[9] T. Wolf et al., 'Transformers: State-of-the-Art Natural Language Processing,' in Proc. EMNLP System Demos, 2020.

[10] D. Jurafsky and J. H. Martin, Speech and Language Processing, 3rd ed. draft, Stanford University, 2024.

[11] H. Zhang et al., 'HaluEval: A Large-Scale Hallucination Evaluation Benchmark for Large Language Models,' in Proc. EMNLP, 2023.

[12] Streamlit Inc., 'Streamlit Documentation,' https://docs.streamlit.io, 2026.